Informe técnico de desarrollo de software: Proyecto TeKnical May Cry

Federico Mattia, Ulises Ferrari, Lucas Del Pino y Máximo Orue

Escuela de Educación Secundaria Técnica N.° 1 “Eduardo Ader”, Vicente López Laboratorio de Programación, 5.° 3.ª A-B

Profesores: York E. Mansilla Muñoz y Yamil Ganduglia 12 de septiembre de 2026

Resumen

Este informe técnico documenta el diseño, la arquitectura, el desarrollo y el control de calidad del proyecto educativo integrador TeKnical May Cry (TmC), realizado durante el ciclo lectivo 2026. El objetivo principal fue unificar las competencias prácticas de la especialidad de programación mediante la construcción de un videojuego bidimensional (2D) en el motor Unity con el lenguaje C#, complementado por una plataforma web oficial maquetada en HTML5, CSS3 y JavaScript, y por un módulo auxiliar escrito en C++. Se describe el flujo de trabajo colaborativo con el sistema de control de versiones Git y GitHub, la organización de archivos y entornos de desarrollo en sistemas Windows y Linux, y el cumplimiento de las normas institucionales de documentación técnica según la séptima edición de las normas APA. Los resultados muestran un sistema funcional, optimizado y documentado.

Palabras clave: C#, Unity 2D, C++, HTML5, Git Bash, normas APA, Laboratorio de Programación

Informe técnico de desarrollo de software: Proyecto TeKnical May Cry

El desarrollo de software en las instituciones de educación técnica requiere la integración práctica de varias disciplinas computacionales. El proyecto TeKnical May Cry (TmC) surge como una respuesta académica orientada a consolidar las habilidades de programación estructurada, programación orientada a objetos (POO), arquitectura de sistemas, diseño web responsivo y gestión de proyectos.

El trabajo se articula en dos vértientes técnicas: por un lado, el desarrollo del motor interactivo y las mecánicas del juego; por el otro, la elaboración de la infraestructura web y de la documentación técnica formal según las normas APA (7.ª ed.), exigida para la entrega en la plataforma Moodle.

Objetivos del proyecto

El objetivo general fue desarrollar un entorno interactivo integrador, compuesto por un videojuego 2D y un portal web técnico, que aplique conceptos de ingeniería de software y programación modular. Los objetivos específicos fueron los siguientes:

1. Diseñar e implementar en C # sobre Unity la lógica de interacción del personaje, los sistemas de cámaras y los minijuegos.
2. Implementar subprogramas auxiliares y algoritmos de cómputo en C++ con Visual Studio Code y el entorno MinGW.
3. Establecer un flujo continuo de control de versiones con Git y GitHub para resolver conflictos en equipo.
4. Redactar el informe técnico y el diario de campo institucional respetando los lineamientos de citación, jerarquía y formato de las normas APA (7.ª ed.).

Marco teórico y estado del arte

Entornos de desarrollo e infiraestructura de compiladores

El desarrollo de software moderno exige distinguir entre los editores de código fuente y los entornos de desarrollo integrados (IDE). Durante el proyecto se utilizó Visual Studio Code como editor liviano y extensible, y Visual Studio Community para la compilación en profundidad de C# (Microsoft, 2023).

Configuración del compilador MinGW y de la extensión C/C++

Para el módulo de lógica CALCULOAREA, escrito en C++, se configuró la suite de compilación MinGW (GCC/G++) asociada a Visual Studio Code. La automatización de la compilación y la depuración se definió mediante el archivo de configuración `tasks.json`

.

Sistemas de archivos y almacenamiento

La gestión del almacenamiento depende de la arquitectura del sistema de archivos subyacente. En Windows se trabajó principalmente sobre NTFS (New Technology File System), que administra sus estructuras mediante la tabla maestra de archivos (MFT, Master File Table) y ofrece registro por diario (*journaling*), permisos propios y asignación por clústeres. Se evaluaron además los sistemas FAT32, exFAT y ext4 para garantizar la portabilidad de los ejecutables compilados de 64 bits.

Metodología y desarrollo del proyecto

Inspiración y diseño conceptual

El videojuego TeKnical May Cry combina mecánicas de dos títulos fundamentales de la industria:

1. **Diseño de mapas y navegación 2D.** Se basó en la estructura cenital (*top-down*) y los mapas en cuadrícula de *Pokémon Rojo Fuego* (Nintendo, 2004). Se emplearon los componentes Tilemap y Tilemap Collider 2D para definir mallas de colisión exactas en salas y pasillos (Unity Technologies, 2025).
2. **Mecánicas de minijuegos y tensión.** Se inspiraron en el ciclo de interacción y el control de cámaras de *Five Nights at Freddy's* (Cawthon, 2014), y se gestionaron mediante clases independientes en C # (`FNaFManager.cs` y `CameraSystem.cs`).

Desarrollo de la lógica en C # y C++

La arquitectura lógica se dividió en módulos independientes para evitar el acoplamiento del código.

Implementación en C # sobre Unity

La lógica del juego administra corrutinas para manejar de forma asíncrona los eventos en pantalla sin bloquear el hilo principal (*main thread*). La Figura 1 muestra la estructura de control del minijuego.

Figura 1

Estructura de control del minijuego en C #

```
public class FNaFManager : MonoBehaviour
{
    public float energyLevel = 100f;

    void Update()
    {
        if (energyLevel > 0)
        {
            energyLevel -= Time.deltaTime * 0.5f;
        }
    }
}
```

Nota. Elaboración propia.

La Figura 2 muestra la terminal de comandos durante la compilación del proyecto.

Figura 2

Ejecución de la terminal de comandos durante la compilación del proyecto

[Insertar aquí la captura de pantalla de la terminal. Nota: Elaboración propia.]

Desarrollo del frontend web

El frontend del portal web oficial se estructuró en HTML5 semántico y se estilizó con hojas de estilo en cascada (CSS3), mediante *flexbox* y grillas adaptativas (*media queries*). La interactividad de la galería y de los visores embebidos de documentos PDF se resolvió con scripts livianos en JavaScript.

Gestión de versiones con Git y GitHub

Se utilizó Git Bash como consola principal para el control de versiones. Se aplicaron flujos de trabajo basados en ramas (*feature branching*) y se resolvieron conflictos de fusión (*merge conflicts*) en los archivos de metadatos `.meta` que genera Unity. Las identidades del equipo se configuraron con credenciales fijas y tokens de acceso personal (PAT).

Resultados y discusión

El desarrollo del proyecto cumplió el 100 % de las especificaciones funcionales programadas en el cronograma. Los resultados principales fueron tres:

1. **Rendimiento.** Se estabilizó la tasa de cuadros por segundo (FPS) del prototipo en Unity al reemplazar llamadas redundantes en el método C # `Update()` por eventos de .
2. **Portabilidad.** Se verificó la compilación del subprograma CALCULOAREA, escrito en C++, en arquitecturas x86 y x64.
3. **Documentación.** Se publicó el informe técnico final y la carpeta de campo en la plataforma Moodle, respetando los estándares de formato (tipografía Arial 11 o Times New Roman 12, interlineado doble y márgenes de 2,54 cm).

Conclusión

El proyecto TeKnical May Cry fue una experiencia integradora y enriquecedora dentro del plan de estudios del ciclo superior de la educación técnica. La combinación de desarrollo de videojuegos en C #, maquetación web y programación estructurada en C++ permitió afianzar los conceptos teóricos en entornos de producción reales.

La superación de obstáculos técnicos, como los errores de referencia nula en los scripts, la configuración de agentes SSH en Git Bash y la maquetación responsiva, fortaleció la autonomía, la perseverancia y la capacidad de resolver problemas en equipo. La integración entre teoría y ejecución reafirma la importancia del modelo de aprendizaje basado en proyectos.

Refierencias

Cawthon, S. (2014). *Five Nights at Freddy's* [Videojuego]. Scott Cawthon.

Microsoft. (2023). *C# documentation: Basics of C# for beginners*. Microsoft Learn. https://learn.microsoft.com/dotnet/csharp/

Nintendo. (2004). *Pokémon Rojo Fuego* [Videojuego]. Nintendo.

Unity Technologies. (2025). *Unity user manual 2022.3 LTS: Tilemaps and Physics 2D*. Unity Documentation. https://docs.unity3d.com/Manual/